Building data course — Week 1 — find your way around (read-only)

sparql-playground / lessons

Generated for print and offline reading

Week 1 — find your way around (read-only)

Five daily lessons. Each day has two scripts that compute the same thing: `dayNN_pandas.py` reads the CSV export into pandas, `dayNN_sql.py` runs SQLite queries on the same rows (`uv run python lessons/course.py setup` loads them once). Run both with `uv run python lessons/course.py run N` and compare the printouts. Each lesson's two scripts are printed after it. Keep notes under the ignored `student-work/` folder.

Contents

- Day 1 — Open the data; what one row is
- Day 2 — SELECT, WHERE, ORDER BY, LIMIT on a few columns
- Day 3 — Time: ISO timestamps, date functions, ranges, local time
- Day 4 — Summary stats: COUNT, MIN, MAX, AVG, SUM
- Day 5 — GROUP BY and HAVING: per day, per month, and the Week 1 challenge

---

Day 1 — Open the data; what one row is

Goal: open AHU_1's export in pandas and in SQLite, and say in one sentence what a row means.

Read

An HVAC controller measures things (temperatures, pressures, fan status) and a **historian** saves a reading every few minutes. Building 50's historian exported one CSV per air handler (AHU): `data/BUILDING_50/AHU_1/history_wide.csv` and the same for `AHU_2`. This week you work with AHU_1, and bring in AHU_2 only where it helps.

The CSV is a **wide** table:

- **One row is one moment**: a timestamp, every five minutes, in UTC.
- **One column is one point**: a sensor, a setpoint, a status or a command. The cell holds that point's value at that moment.

So `timestamp_utc = 2026-03-16 00:40:00+00:00` and `outside_air_temp_f = 69.0327` in the same row means: at 00:40 UTC the outside air sensor read 69.0327 °F. AHU_1's file has 84 columns (the timestamp plus 83 points) and 35,536 rows. (The opposite shape, one row per point per timestamp, is called **long**. Day 11 converts between the two, and Week 5's API stores long data.)

Two scripts open the same data two ways:

	`day01_pandas.py`	`day01_s`
open	`pd.read_csv(".../AHU_1/history_wide.csv")`	the table ah `var/lesson`
how many rows	`len(df)` / `df.shape[0]`	`SELECT cou FROM ahu_1`
what are the	`df.columns`, `df.dtypes`	`PRAGMA table_info`

	day01_pandas.py	day01_s
columns		
first rows	`df[cols].head(5)`	`SELECT col ahu_1 ORDE timestamp_ LIMIT 5`

What `setup_db.py` does. `uv run python lessons/course.py setup` runs `lessons/setup_db.py`, which reads each CSV and writes one SQLite table: `ahu_1`, `ahu_2` (and `fdd_events`, which Day 14 uses). Nothing is transformed. The columns keep their names and their order, `timestamp_utc` is stored as the same text as in the file and is the `PRIMARY KEY`, and every other column is `REAL`. So the pandas script and the SQL script look at identical rows. SQLite is a whole SQL database in one file: no server, no login, and Python ships with it (`import sqlite3`).

What the columns mean. Next to each CSV sits `columns.csv`: one line per column with the historian's point name (`bld_50_AHU_1 AHU1 OA-T`), its unit and a rough role. Names like `outside_air_temp_f` are cleaned-up versions of those. Units are US units: °F, inches of water (`inwc`), percent 0–100, and statuses are 0 or 1 stored as numbers. `columns.csv` is a file, not a table in the database, so only the pandas script reads it (step 6).

Why the column names are typed into code. All of Weeks 1–4 write names like `outside_air_temp_f` and `supply_fan_status` straight into the scripts. That is how a first analysis looks, and it is fine for one building. It also means the code only works on exports that use exactly those names: AHU_2's file has 67 columns, and calls its outside-air damper `mad_c_pct` where AHU_1 has `mad_c`. Week 5 replaces typed names with a model that says which column is the duct static pressure, so the same rule runs on any air handler.

Run

```
uv run python lessons/course.py setup        # once
uv run python lessons/course.py run 1
```

That runs `day01_pandas.py`, then `day01_sql.py`. Read them side by side: each numbered comment in one has the same number in the other. Both print the same shape, column names, first rows and time span:

```
rows x columns per AHU:
equipment_id  rows  columns
       AHU_1 35536       84
       AHU_2 35536       67

the first five rows, five columns:
            timestamp_utc  outside_air_temp_f  da_p_inwc
da_p_setpoint_inwc  supply_fan_status
2026-03-16 00:40:00+00:00             69.0327   0.024594
1.2                0.0
...

time span:
                first_utc                  last_utc
distinct_timestamps
2026-03-16 00:40:00+00:00 2026-07-17 09:55:00+00:00
35536
```

35,536 rows × 5 minutes is about 123 days, and 35,536 distinct timestamps means every row is its own moment. pandas reports dtype `float64` for the 83 points and `object` (text) for the timestamp; SQLite reports declared types `REAL` and `TEXT`. The same fact in each tool's vocabulary.

Two things to notice:

- `supply_fan_status` prints as `0.0`, not `0`. A status is a number in a historian, and the CSV stores it as a decimal.
- `read_csv` pulled all 84 columns into memory (about 24 MB). Fine here. From Day 2 on the scripts load only the columns they use (`usecols=[...]`).

Try it

1. How many columns does AHU_2 share with AHU_1? Which non-zone columns does only one of them have?
2. How many of AHU_1's columns are zone temperatures (names starting `zone_t50_`)?

Answers

1. 34 columns are common, 50 are only in AHU_1 and 33 only in AHU_2. Outside the zone temperatures, AHU_1 alone has `50ahu1aalrm`, `mad_c` and a long auto-generated `hu_50_duration_...` column; AHU_2 alone has `50ahu2aalrm` and `mad_c_pct`. The `mad_c`/`mad_c_pct` pair is the same kind of point (outside air damper) under two names: a rule written with a typed column name would work on one AHU and not the other.

   ```
   a = set(pd.read_csv(DATA / "AHU_1" /
             "history_wide.csv", nrows=0).columns)  #
             nrows=0: header only
   b = set(pd.read_csv(DATA / "AHU_2" /
             "history_wide.csv", nrows=0).columns)
   print(len(a & b), sorted(a - b), sorted(b - a))
   ```

   In SQL the same comparison is `SELECT name FROM pragma_table_info('ahu_1') EXCEPT SELECT name FROM pragma_table_info('ahu_2')`.

2. 47 of the 84 (31 in AHU_2). In pandas: `sum(c.startswith("zone_t50_") for c in df.columns)`; in SQL, count the rows of `pragma_table_info('ahu_1')` whose `name LIKE 'zone%'`. Most of an AHU's columns are readings from the zones it serves, not from the AHU itself.

Why it matters

Every analysis starts with "what is one row, and what is one column?". Here a row is a moment and a column is a point, which makes questions about one moment easy (compare the fan and the duct static in the same row) and questions about one point easy (take a column). What this table does not say is what each column *is*: `da_p_inwc` is the duct static pressure only because `columns.csv`, and some building knowledge, say so. Closing that gap is the point of Week 5.

day01_pandas.py

```python
"""Day 1, pandas: open AHU_1's CSV export and see what one
        row is.

    uv run python lessons/unit1-sqlite/day01_pandas.py

The historian's export is a "wide" table: one row per
        timestamp, one column per point.
read_csv gives it to you as a DataFrame; shape, columns and
        dtypes say what you are holding.
Compare the printout with day01_sql.py: the numbers should
        match.
"""

from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"

# 1. Open AHU_1: every column (usecols is for later days;
        today the point is to see them all).
ahu_1 = pd.read_csv(DATA / "AHU_1" / "history_wide.csv")

# 2. Shape: (rows, columns). AHU_2 is a different air
        handler with a different set of points.
ahu_2 = pd.read_csv(DATA / "AHU_2" / "history_wide.csv")
shapes = pd.DataFrame(
    {
        "equipment_id": ["AHU_1", "AHU_2"],
        "rows": [len(ahu_1), len(ahu_2)],
        "columns": [ahu_1.shape[1], ahu_2.shape[1]],
    }
)

# 3. Columns: position and name (the DataFrame's version of
        PRAGMA table_info).
columns = pd.DataFrame({"cid": range(ahu_1.shape[1]),
        "name": ahu_1.columns})
dtype_counts = ahu_1.dtypes.astype(
    str
).value_counts()  # timestamp is text (object), the rest
        float64

# 4. The first rows, a few columns. One row = one moment;
        each column = one point's value then.
SHOW = [
    "timestamp_utc",
```

```
    "outside_air_temp_f",
    "da_p_inwc",
    "da_p_setpoint_inwc",
    "supply_fan_status",
]
first_rows = ahu_1[SHOW].head(5)

# 5. One row per timestamp: no duplicates, and the time
     span the export covers.
time_span = pd.DataFrame(
    {
        "first_utc": [ahu_1["timestamp_utc"].min()],
        "last_utc": [ahu_1["timestamp_utc"].max()],
        "distinct_timestamps":
         [ahu_1["timestamp_utc"].nunique()],
    }
)

# 6. What each column is: columns.csv sits next to the
     history (it is not loaded into SQLite).
meta = pd.read_csv(DATA / "AHU_1" / "columns.csv")
described = meta[meta["col"].isin(SHOW[1:])][["col",
     "point_name"]]

RESULTS = {
    "shapes": shapes,
    "columns": columns,
    "first_rows": first_rows,
    "time_span": time_span,
}

print("1. rows x columns per AHU:")
print(shapes.to_string(index=False))
print("\n2. AHU_1 dtypes (how many columns of each kind):")
print(dtype_counts.to_string())
print("\n3. the first and last five column names:")
print(columns.iloc[[*range(5), *range(-5,
     0)]].to_string(index=False))
print("\n4. the first five rows, five columns:")
print(first_rows.to_string(index=False))
print("\n5. time span:")
print(time_span.to_string(index=False))
print("\n6. what the columns mean (from columns.csv):")
print(described.to_string(index=False))
```

day01_sql.py

```
"""Day 1, SQL: the same AHU_1 export, loaded as the SQLite
        table `ahu_1`.

    uv run python lessons/course.py setup         # once:
        loads the CSVs into var/lessons.sqlite
    uv run python lessons/unit1-sqlite/day01_sql.py

setup_db.py made one table per CSV: same column names, same
        values, nothing transformed. This script
asks SQLite what it holds. SQLite does all the work; pandas
        only turns the result rows into a table
for printing.
Compare the printout with day01_pandas.py: the numbers
        should match.
"""

import os
import sqlite3
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DB = Path(os.environ.get("LESSONS_DB", ROOT / "var" /
        "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv run python
        lessons/course.py setup")
conn = sqlite3.connect(DB)

# 1. The table's structure comes from SQLite's own catalog.
        PRAGMA table_info lists the columns.
TABLE_INFO_SQL = "PRAGMA table_info(ahu_1)"  # cid, name,
        declared type, notnull, default, pk

# 2. Shape: count(*) rows, and count the rows PRAGMA
        table_info would return for the column count.
SHAPES_SQL = """
SELECT 'AHU_1' AS equipment_id,
       (SELECT count(*) FROM ahu_1) AS rows,
       (SELECT count(*) FROM pragma_table_info('ahu_1')) AS
         columns
UNION ALL
SELECT 'AHU_2',
       (SELECT count(*) FROM ahu_2),
       (SELECT count(*) FROM pragma_table_info('ahu_2'))
"""
```

```
# 3. Columns: position and name. pragma_table_info() is
        PRAGMA table_info as a selectable table.
COLUMNS_SQL = "SELECT cid, name FROM
        pragma_table_info('ahu_1') ORDER BY cid"
TYPE_COUNTS_SQL = "SELECT type, count(*) AS columns FROM
        pragma_table_info('ahu_1') GROUP BY type"

# 4. The first rows, a few columns. Without ORDER BY SQL
        promises no order, so sort by the key.
FIRST_ROWS_SQL = """
SELECT timestamp_utc, outside_air_temp_f, da_p_inwc,
        da_p_setpoint_inwc, supply_fan_status
FROM ahu_1
ORDER BY timestamp_utc
LIMIT 5
"""

# 5. One row per timestamp: the PRIMARY KEY forbids
        duplicates; this counts them anyway.
TIME_SPAN_SQL = """
SELECT min(timestamp_utc) AS first_utc,
       max(timestamp_utc) AS last_utc,
       count(DISTINCT timestamp_utc) AS distinct_timestamps
FROM ahu_1
"""

table_info = pd.read_sql_query(TABLE_INFO_SQL, conn)
shapes = pd.read_sql_query(SHAPES_SQL, conn)
columns = pd.read_sql_query(COLUMNS_SQL, conn)
type_counts = pd.read_sql_query(TYPE_COUNTS_SQL, conn)
first_rows = pd.read_sql_query(FIRST_ROWS_SQL, conn)
time_span = pd.read_sql_query(TIME_SPAN_SQL, conn)

# 6. columns.csv (what each column means) is a file next to
        the history, not a table here.

RESULTS = {
    "shapes": shapes,
    "columns": columns,
    "first_rows": first_rows,
    "time_span": time_span,
}

print("1. PRAGMA table_info(ahu_1), some rows (cid, name,
        declared type, notnull, default, pk):")
print(table_info.iloc[[0, 1, 2, -2,
        -1]].to_string(index=False))
print("\n2. rows x columns per AHU:")
print(shapes.to_string(index=False))
print("\n   declared types (how many columns of each
        kind):")
```

```
print(type_counts.to_string(index=False))
print("\n3. the first and last five column names:")
print(columns.iloc[[*range(5), *range(-5,
        0)]].to_string(index=False))
print("\n4. the first five rows, five columns:")
print(first_rows.to_string(index=False))
print("\n5. time span:")
print(time_span.to_string(index=False))
```

Day 2 — SELECT, WHERE, ORDER BY, LIMIT on a few columns

Goal: pull a few columns of AHU_1, filtered, sorted and trimmed, in pandas and in SQL, and pass values to SQL safely.

Read

The parts of a query, in the order you write them, and the pandas that does each job:

```
SELECT  columns            -- what to show
df[["timestamp_utc", "outside_air_temp_f"]]
FROM    table              -- where from              (the
DataFrame you loaded)
WHERE   condition          -- which rows
df[df.da_p_inwc > 1.5]
ORDER BY column [DESC]     -- sorted how
.sort_values("outside_air_temp_f", ascending=False)
LIMIT   n                  -- how many
.head(n)
```

Today's columns, all from AHU_1's table `ahu_1`:

Column	What it is	Unit
`outside_air_temp_f`	outside air temperature	°F
`da_p_inwc`	duct static pressure (the pressure the supply fan builds in the supply duct)	inches of water
`da_p_setpoint_inwc`	the pressure the controller is trying to hold	inches of water
`supply_fan_status`	1 when the supply fan is running, 0 when off	0/1

Text values go in single quotes in SQL. Double quotes are for column and table names. To combine conditions use `AND` / `OR` in SQL and `&` / `|` in pandas. Each condition needs its own

parentheses in pandas, because `&` binds tighter than `>`.

Parameters, not f-strings. In Python you'll be tempted to build SQL with an f-string: `f"... WHERE timestamp_utc = '{ts}'"`. Don't. If `ts` ever comes from a user, a file or an API request, a value like `x' OR '1'='1` rewrites your query (**SQL injection**), and even with trusted values you get quoting bugs. Instead leave a placeholder, `?` or a name like `:limit`, and pass the value separately. The database receives the SQL text and the value as two different things, so a value can never be mistaken for SQL. It also lets SQLite reuse the compiled query. Every query in this course passes values as parameters; only column and table names (which cannot be parameters) are written into the SQL text, and those are typed by you, never taken from input.

Where the work happens. `timestamp_utc` is the table's `PRIMARY KEY`, and SQLite keeps an index on it: a sorted B-tree. So `WHERE timestamp_utc >= :start AND timestamp_utc < :end` is a **seek**: walk the tree from the root to the first matching key, O(log n), then read the k matching rows in order. pandas has no such luxury on a plain column. `df["timestamp_utc"] >= start` compares every one of the n rows and builds a True/False array of length n: O(n). (A DataFrame whose index is sorted can do binary search too, but you pay for the sort, and the rows are already in memory.) A filter on `da_p_inwc` has no index in either tool, so both scan all n rows. `EXPLAIN QUERY PLAN` shows which one SQLite chose; the SQL script prints it.

A sort is different again: `ORDER BY ... LIMIT 1` only needs the biggest row, and `sort_values(...).head(1)` sorts all n rows to get it, O(n log n), where a single pass for the maximum would do. (`nlargest(1)` is the pandas way to ask for just that.) At 35,000 rows nobody notices. At a hundred million rows it is the whole job.

Run

```
uv run python lessons/course.py run 2
```

That runs `day02_pandas.py`, then `day02_sql.py`. Each step maps like this:

Step	day02_sql.py	day02_pandas.py
pick columns	`SELECT timestamp_utc, outside_air_temp_f`	`df[["timestamp_utc", "outside_air_temp_f"]]`
first three	`ORDER BY timestamp_utc LIMIT 3`	`.sort_values("timestamp_utc").h`
hottest	`ORDER BY outside_air_temp_f DESC, timestamp_utc LIMIT 1`	`sort_values([...], ascending=[F True]).head(1)`
filter + count	`WHERE da_p_inwc > :limit` + `count(*)`	`(df["da_p_inwc"] > 1.5).sum()`
two conditions	`WHERE a = 1 AND b > :min_oat`	`df[(df.a == 1) & (df.b > 95.0)]`
time range	`WHERE timestamp_utc >= :start AND timestamp_utc < :end`	two comparisons joined by `&`
distinct values	`SELECT DISTINCT supply_fan_status`	`df["supply_fan_status"].unique(`

In pandas, `df["da_p_inwc"] > 1.5` makes a column of True/False, one per row, and `.sum()` counts the Trues. SQL's `WHERE` drops the rows and `count(*)` counts what is left. Same answer; SQL never builds the 35,536-element mask.

Adding `timestamp_utc` as a second sort key makes "hottest" deterministic: several readings can tie, because the sensor value only changes now and then, and without a tiebreaker SQL and pandas are free to return different ones.

Both scripts print:

```
hottest / coldest reading:
            timestamp_utc  outside_air_temp_f
2026-07-14 21:40:00+00:00               98.86
2026-03-17 09:40:00+00:00               17.83

readings with duct static above 1.5 inH2O: 8336

readings on 2026-07-01 (UTC): 288
rows matching a hostile string: 0
fan status takes the values: [0.0, 1.0]
```

The SQL script also prints SQLite's plan for the date range: `SEARCH ahu_1 USING COVERING INDEX sqlite_autoindex_ahu_1_1 (timestamp_utc>? AND timestamp_utc<?)`. That `SEARCH ... (timestamp_utc>? AND timestamp_utc<?)` is the B-tree seek. The index has an odd name because `PRIMARY KEY` on a text column makes SQLite build an automatic index. 288 is 24 hours × 12 readings an hour: a full day of 5-minute data.

The range query works on text, by luck. `timestamp_utc` is text, and every value has the same format (`2026-07-01 00:00:00+00:00`), so comparing the strings alphabetically is the same as comparing the times. That is the whole reason the bounds in the scripts are written out in full. Day 3 looks at what goes wrong with a shorter bound like `'2026-07-01'`, and at the proper date functions.

The hostile string `x' OR '1'='1` is passed as the `:ts` parameter. It is treated as a plain timestamp that matches no row, so the count is 0. Pasted into the SQL with an f-string, it would have turned the `WHERE` into "always true" and counted all 35,536 rows.

Try it

1. What was the coldest outside air reading, and when?
2. How many readings have the fan running (`supply_fan_status = 1`) but duct static below 0.5 inH2O? (Both scripts pass the limit as a variable or parameter.)
3. Print the five most recent timestamps, newest first.

Answers

1. 17.83 °F at `2026-03-17 09:40:00+00:00`. It is already in the printout (`ORDER BY outside_air_temp_f, timestamp_utc LIMIT 1`).

2. 782 readings, about 65 fan-running hours at pressure well below any normal setpoint (a hint of Day 14's FC1 rule).

```
n = int(((df["supply_fan_status"] == 1) &
         (df["da_p_inwc"] < 0.5)).sum())
```

```
conn.execute(
    "SELECT count(*) FROM ahu_1 WHERE supply_fan_status
         = 1 AND da_p_inwc < :limit", {"limit": 0.5}
).fetchone()
```

3. Flip the sort: `ORDER BY timestamp_utc DESC LIMIT 5` in SQL, `df["timestamp_utc"].sort_values(ascending=False).head(5)` in pandas (or just `df["timestamp_utc"].tail(5)`) in pandas. The newest row is `2026-07-17 09:55:00+00:00`. SQL's `DESC` on the primary key reads the B-tree backwards: no sort at all.

Why it matters

Almost every analysis step is "some columns, some rows, in some order, not too many". Getting comfortable with that shape in both tools means you can read other people's queries and DataFrame code as the same thing. Passing values as parameters is a habit worth forming now: in Week 5 the column and series you ask for will come from a model lookup, not from your own keyboard, and a value that came from somewhere else must never be able to become part of the query.

day02_pandas.py

```python
"""Day 2, pandas: pick columns, filter rows, sort and trim,
        on AHU_1.

    uv run python lessons/unit1-sqlite/day02_pandas.py

SELECT = df[[...]], WHERE = a boolean mask, ORDER BY =
        sort_values, LIMIT = head.
Column names are hard-coded on purpose: this is how a first
        FDD script looks. (Week 5 replaces
them with a model lookup.)
Compare the printout with day02_sql.py: the numbers should
        match.
"""

from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"

# 1. Load only the columns today's questions use.
COLUMNS = [
    "timestamp_utc",
    "outside_air_temp_f",
    "da_p_inwc",
    "da_p_setpoint_inwc",
    "supply_fan_status",
]
df = pd.read_csv(DATA / "AHU_1" / "history_wide.csv",
        usecols=COLUMNS)

# 2. SELECT a few columns, ORDER BY time, LIMIT 3: the
        first three outside air readings.
first3 = (
    df[["timestamp_utc",
        "outside_air_temp_f"]].sort_values("timestamp_utc").head(3)
).reset_index(drop=True)

# 3. Hottest and coldest reading: sort by value, keep one
        row. The second sort key breaks ties.
hottest = (
    df[["timestamp_utc", "outside_air_temp_f"]]
    .sort_values(["outside_air_temp_f", "timestamp_utc"],
        ascending=[False, True])
    .head(1)
    .reset_index(drop=True)
)
```

```
coldest = (
    df[["timestamp_utc", "outside_air_temp_f"]]
    .sort_values(["outside_air_temp_f", "timestamp_utc"])
    .head(1)
    .reset_index(drop=True)
)

# 4. WHERE with a limit that is a variable (SQL binds it as
        a parameter): static above 1.5 inH2O.
LIMIT_INWC = 1.5
high_static = pd.DataFrame({"n": [int((df["da_p_inwc"] >
        LIMIT_INWC).sum())]})

# 5. Two conditions: fan running AND hot outside. Hottest
        five such rows.
MIN_OAT_F = 95.0
hot_running = (
    df[(df["supply_fan_status"] == 1) &
        (df["outside_air_temp_f"] > MIN_OAT_F)][
       ["timestamp_utc", "outside_air_temp_f",
        "da_p_inwc"]
    ]
    .sort_values(["outside_air_temp_f", "timestamp_utc"],
        ascending=[False, True])
    .head(5)
    .reset_index(drop=True)
)

# 6. A time range. The timestamps are ISO text in one
        format, so string comparison is time order.
START, END = "2026-07-01 00:00:00+00:00", "2026-07-02
        00:00:00+00:00"
july_1 = pd.DataFrame(
    {"n": [int(((df["timestamp_utc"] >= START) &
        (df["timestamp_utc"] < END)).sum())]}
)

# 7. Hostile input is just a string that matches nothing
        (SQL: it travels as a parameter).
EVIL = "x' OR '1'='1"
hostile = pd.DataFrame({"n": [int((df["timestamp_utc"] ==
        EVIL).sum())]})

# 8. Which values does a status point take? (DISTINCT)
fan_values = pd.DataFrame({"fan_status":
        sorted(df["supply_fan_status"].unique())})

RESULTS = {
    "first3": first3,
    "hottest": hottest,
    "coldest": coldest,
```

```
    "high_static": high_static,
    "hot_running": hot_running,
    "july_1": july_1,
    "hostile": hostile,
    "fan_values": fan_values,
}

print("first three outside air readings:")
print(first3.to_string(index=False))
print("\nhottest / coldest reading:")
print(pd.concat([hottest,
         coldest]).round(2).to_string(index=False))
print(f"\nreadings with duct static above {LIMIT_INWC}
         inH2O:", high_static["n"][0])
print(f"\nhottest five readings above {MIN_OAT_F} F with
         the fan on:")
print(hot_running.round(3).to_string(index=False))
print("\nreadings on 2026-07-01 (UTC):", july_1["n"][0])
print("rows matching a hostile string:", hostile["n"][0])
print("fan status takes the values:",
         fan_values["fan_status"].tolist())
```

day02_sql.py

```
"""Day 2, SQL: SELECT, WHERE, ORDER BY, LIMIT on a few
        AHU_1 columns, values as parameters.

    uv run python lessons/course.py setup         # once:
        loads the CSVs into var/lessons.sqlite
    uv run python lessons/unit1-sqlite/day02_sql.py

The same questions as day02_pandas.py, answered by SQLite
        on the same rows. SQLite does all the
work; pandas only turns the result rows into a table for
        printing.
Every value reaches SQL as a named parameter (:limit),
        never pasted in with an f-string.
Compare the printout with day02_pandas.py: the numbers
        should match.
"""

import os
import sqlite3
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DB = Path(os.environ.get("LESSONS_DB", ROOT / "var" /
        "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv run python
        lessons/course.py setup")
conn = sqlite3.connect(DB)


def q(sql, params=None):
    """Run one SELECT with its parameters and return a
        DataFrame."""
    return pd.read_sql_query(sql, conn, params=params or
        {})


# 1. The table is `ahu_1` (the CSV, unchanged). SELECT
        names the columns; FROM names the table.

# 2. SELECT a few columns, ORDER BY time, LIMIT 3: the
        first three outside air readings.
FIRST3_SQL = """
SELECT timestamp_utc, outside_air_temp_f
FROM ahu_1
ORDER BY timestamp_utc
LIMIT 3
```

```
"""

# 3. Hottest and coldest reading: sort by value, keep one
        row. The second sort key breaks ties.
HOTTEST_SQL = """
SELECT timestamp_utc, outside_air_temp_f
FROM ahu_1
ORDER BY outside_air_temp_f DESC, timestamp_utc
LIMIT 1
"""
COLDEST_SQL = """
SELECT timestamp_utc, outside_air_temp_f
FROM ahu_1
ORDER BY outside_air_temp_f, timestamp_utc
LIMIT 1
"""

# 4. WHERE with the limit bound as a parameter: static
        above 1.5 inH2O.
HIGH_STATIC_SQL = "SELECT count(*) AS n FROM ahu_1 WHERE
        da_p_inwc > :limit"

# 5. Two conditions: fan running AND hot outside. Hottest
        five such rows.
HOT_RUNNING_SQL = """
SELECT timestamp_utc, outside_air_temp_f, da_p_inwc
FROM ahu_1
WHERE supply_fan_status = 1 AND outside_air_temp_f >
        :min_oat
ORDER BY outside_air_temp_f DESC, timestamp_utc
LIMIT 5
"""

# 6. A time range. timestamp_utc is the PRIMARY KEY, so
        SQLite finds `>= :start` with a B-tree
# seek and reads on to `< :end`: O(log n + k). A pandas
        mask compares every row: O(n).
JULY_1_SQL = """
SELECT count(*) AS n FROM ahu_1
WHERE timestamp_utc >= :start AND timestamp_utc < :end
"""

# 7. Hostile input is just a string that matches nothing:
        it travels beside the SQL, not inside it.
HOSTILE_SQL = "SELECT count(*) AS n FROM ahu_1 WHERE
        timestamp_utc = :ts"

# 8. Which values does a status point take? (DISTINCT)
FAN_VALUES_SQL = """
SELECT DISTINCT supply_fan_status AS fan_status FROM ahu_1
        ORDER BY fan_status
```

```
"""

START, END = "2026-07-01 00:00:00+00:00", "2026-07-02
        00:00:00+00:00"
first3 = q(FIRST3_SQL)
hottest = q(HOTTEST_SQL)
coldest = q(COLDEST_SQL)
high_static = q(HIGH_STATIC_SQL, {"limit": 1.5})
hot_running = q(HOT_RUNNING_SQL, {"min_oat": 95.0})
july_1 = q(JULY_1_SQL, {"start": START, "end": END})
hostile = q(HOSTILE_SQL, {"ts": "x' OR '1'='1"})
fan_values = q(FAN_VALUES_SQL)

# What did SQLite plan for the time range? SEARCH ... USING
        INDEX is a seek; SCAN reads every row.
plan = conn.execute("EXPLAIN QUERY PLAN " + JULY_1_SQL,
        {"start": START, "end": END}).fetchall()

RESULTS = {
    "first3": first3,
    "hottest": hottest,
    "coldest": coldest,
    "high_static": high_static,
    "hot_running": hot_running,
    "july_1": july_1,
    "hostile": hostile,
    "fan_values": fan_values,
}

print("first three outside air readings:")
print(first3.to_string(index=False))
print("\nhottest / coldest reading:")
print(pd.concat([hottest,
        coldest]).round(2).to_string(index=False))
print("\nreadings with duct static above 1.5 inH2O:",
        high_static["n"][0])
print("\nhottest five readings above 95.0 F with the fan
        on:")
print(hot_running.round(3).to_string(index=False))
print("\nreadings on 2026-07-01 (UTC):", july_1["n"][0])
print("query plan for that range:", plan[0][-1])
print("rows matching a hostile string:", hostile["n"][0])
print("fan status takes the values:",
        fan_values["fan_status"].tolist())
```

Day 3 — Time: ISO timestamps, date functions, ranges, local time

Goal: turn `timestamp_utc` into dates and date parts, select a day, and know where each tool's time handling bites.

Read

`timestamp_utc` is **ISO 8601 text**: `2026-03-16 00:40:00+00:00` is year-month-day, a space, hour:minute:second, and a UTC offset (`+00:00` means UTC). Historians store UTC because it has no daylight saving jumps and sorts the same way as the clock.

Because every value has the same fixed-width format, **sorting the text is sorting by time**. That is why Day 2's range query worked on raw text. But text is still text: to get *parts* of the date (the hour, the weekday, the month) or to do arithmetic, you need real date handling:

- **pandas** parses once: `df["time"] = pd.to_datetime(df["timestamp_utc"], utc=True)` gives a datetime column (`datetime64[ns, UTC]`) with the whole `.dt` toolbox.
- **SQLite** has no date type at all. `datetime()`, `date()`, `strftime()` and `unixepoch()` read the text and compute on it **row by row**, every time you call them.

How the two line up for `2026-03-16 00:40:00+00:00`:

SQL (on the text column)	pandas (`t = pd.to_datetime(..., utc=True)`)	Result
`datetime(timestamp_utc)`	`t.dt.strftime("%Y-%m-%d %H:%M:%S")`	`2026-03-1 00:40:00` (offset dropped)
`date(timestamp_utc)`	`t.dt.strftime("%Y-%m-%d")`	`2026-03-1`

SQL (on the text column)	pandas (`t = pd.to_datetime(..., utc=True)`)	Result
`strftime('%H', timestamp_utc)`	`t.dt.hour`	`00` (text in SQL, int in pandas)
`strftime('%w', timestamp_utc)`	`(t.dt.dayofweek + 1) % 7`	`1` (a Monday)
`unixepoch(timestamp_utc)`	`(t - epoch).dt.total_seconds()`	`177362160`

`unixepoch` is **Unix epoch seconds**: seconds since 1970-01-01 00:00:00 UTC. Subtracting two of them gives seconds between readings, which is how the scripts check there are no gaps: 5-minute steps from the first to the last reading need `span_s / 300 + 1 = 35,536` rows, and there are exactly that many.

Weekday gotcha. SQLite's `%w` counts **Sunday = 0** and returns **text**; pandas' `dayofweek` counts **Monday = 0** and returns an integer. 2026-03-16 was a Monday: 1 in SQL, 0 in pandas. The pandas script converts with `(dayofweek + 1) % 7` before comparing, and the SQL script wraps `%w` in `CAST(... AS INTEGER)`.

Date ranges: half-open. To select a day, write `start <= time < end` with `end` = the next midnight. Consecutive days then never share a reading, and "the day" needs no thought about 23:59:59.999. Closed ranges (`BETWEEN`, `<=`) are where off-by-one days come from. They also bite on short text: `BETWEEN '2026-07-01' AND '2026-07-01'` returns **zero** rows, because `'2026-07-01 00:00:00+00:00'` sorts *after* `'2026-07-01'` (a longer string with the same start is greater). The scripts build bounds as full timestamps, `2026-07-01 00:00:00+00:00`, which in pandas is simply `str(pd.Timestamp("2026-07-01", tz="UTC"))`.

Don't filter with a function of the column. `WHERE date(timestamp_utc) = '2026-07-01'` returns the same 288 rows as the range, but SQLite has to compute a date for every row instead of seeking. The range form lets it use the index: a B-tree seek, O(log n + k). The function form is a scan, O(n). The plans, printed by the SQL script:

```
SEARCH ahu_1 USING COVERING INDEX sqlite_autoindex_ahu_1_1
(timestamp_utc>? AND timestamp_utc<?)
SCAN ahu_1 USING COVERING INDEX sqlite_autoindex_ahu_1_1
```

(Both read the index, since it holds the whole key; `SEARCH` jumps to the range, `SCAN` walks all of it.) The pandas twin of the mistake is formatting the whole column as strings just to compare dates, which is a Python-level pass over every row; comparing the datetime column to two bounds is one vectorized comparison. With 35,000 rows both are instant; with a month of 1-second data from a thousand points the difference is minutes.

Time zones. Store UTC, convert at the edges. pandas converts to a named zone, daylight saving included: `t.dt.tz_convert("America/Chicago")`. SQLite has only `'localtime'`, the zone of **whatever machine runs the query**, so the same query gives different answers on different computers. That is a reason to keep zone conversion in pandas or application code. Building 50 is in the Central zone. Its data (March to July) is all daylight saving time, UTC−5; the spring change was 8 March, before the data starts, and the autumn one is in November, after it ends.

Run

```
uv run python lessons/course.py run 3
```

That runs `day03_pandas.py`, then `day03_sql.py`. Both print:

```
first three readings, as dates and epoch seconds:
           time_utc    epoch_s  outside_air_temp_f
2026-03-16 00:40:00 1773621600             69.0327
...
first and last timestamp:
          first_utc            last_utc
2026-03-16 00:40:00 2026-07-17 09:55:00
rows if every 5-minute step is present: 35536
rows actually there:                    35536

readings on 2026-07-01 (half-open range): 288
readings on 2026-03-16 (data starts at 00:40): 280
same day via a computed date on every row: 288
BETWEEN '2026-07-01' AND '2026-07-01' on text: 0 (the day's
rows all sort after it)

weekday of the first row (0 = Sunday, as SQL counts): 1
the hottest reading, UTC: 2026-07-14 21:40:00 98.86
```

288 = 24 hours × 12 readings an hour: a full day of 5-minute data. The first day, 2026-03-16, has 280, because the data starts at 00:40, not midnight, so eight readings never logged.

After the shared results each script adds its own tail, because they differ:

- pandas converts to local time: `2026-07-14 16:40:00-05:00` for the hottest reading, and the fan's first start on 2026-04-01 was `09:00 UTC` = **04:00 local**. It also shows pandas refusing to compare a timezone-naive column with an aware timestamp (`TypeError: Invalid comparison...`), the classic pandas time bug. That error is the friendly outcome: the unfriendly one is a library that silently compares and is off by five hours. Pass `utc=True` when you parse, and build bounds with `tz="UTC"`.
- SQL prints its query plans and `datetime(..., 'localtime')`, which gives the right answer on a machine set to Central time and a different one anywhere else.

Try it

1. When was the hottest reading in `America/Chicago` time, and is that a plausible hour for a hot day?

2. How many readings does June 2026 have, as a half-open range? Why would `<= '2026-06-30'` be wrong?
3. When did the fan first start on 2026-06-04, in UTC and in local time? (Copy the fan-start step; change the dates.)

Answers

1. 16:40 (4:40 pm) local on 14 July, `21:40 UTC` minus five hours. Mid-afternoon is when the outdoor temperature peaks, so yes. (pandas only: `hot_time.tz_convert("America/Chicago")`.)
2. 8,640 (30 days × 288), with `timestamp_utc >= '2026-06-01 00:00:00+00:00' AND timestamp_utc < '2026-07-01 00:00:00+00:00'`. `<= '2026-06-30'` compares the *text*, and every reading on the 30th (`2026-06-30 00:05:00+00:00` and so on) sorts after `'2026-06-30'`, so the whole last day would be missing.
3. 09:05 UTC, which is 04:05 local (UTC–5). On both mornings checked the fan starts around 4 am local: an early start (a warm-up or cool-down before occupancy?), the kind of thing you only see once you read the time in local terms.

Why it matters

Building people think in local time ("the fan should start at 6 am"). Databases should store UTC. Converting at the edges and noticing what a UTC timestamp is in local time is a daily skill: a schedule fault looks like a five-hour mistake if you forget. The half-open range and the "never wrap the indexed column in a function" rules are the two habits from today that you will use in every later week.

day03_pandas.py

```python
"""Day 3, pandas: time. ISO text timestamps, date parts,
        half-open ranges, local time.

    uv run python lessons/unit1-sqlite/day03_pandas.py

The CSV's timestamp_utc is text like '2026-03-16
        00:40:00+00:00'. pd.to_datetime turns it into a
real time-zone-aware datetime column with the whole .dt
        toolbox.
Compare the printout with day03_sql.py: the numbers should
        match.
"""

from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"
FMT = "%Y-%m-%d %H:%M:%S"
EPOCH = pd.Timestamp("1970-01-01", tz="UTC")

# 1. Load, then parse the text into datetime64[ns, UTC].
        utc=True keeps every value time-zone aware.
COLUMNS = ["timestamp_utc", "outside_air_temp_f",
        "supply_fan_status"]
df = pd.read_csv(DATA / "AHU_1" / "history_wide.csv",
        usecols=COLUMNS)
df["time"] = pd.to_datetime(df["timestamp_utc"], utc=True)

# 2. The first three readings as dates and as Unix epoch
        seconds (seconds since 1970-01-01 UTC).
first3 = df.sort_values("time").head(3)
first3 = pd.DataFrame(
    {
        "time_utc": first3["time"].dt.strftime(FMT),
        "epoch_s": (first3["time"] -
         EPOCH).dt.total_seconds().astype(int),
        "outside_air_temp_f": first3["outside_air_temp_f"],
    }
).reset_index(drop=True)

# 3. First and last timestamp, and a gap check: 5-minute
        steps from first to last need N rows.
first, last = df["time"].min(), df["time"].max()
span = pd.DataFrame({"first_utc": [first.strftime(FMT)],
        "last_utc": [last.strftime(FMT)]})
span_s = int((last - first).total_seconds())
```

```
gap_check = pd.DataFrame(
    {"span_s": [span_s], "expected_rows": [span_s // 300 +
        1], "actual_rows": [len(df)]}
)

# 4. A day, as a HALF-OPEN range: start <= time < end, so
        consecutive days never share a reading.
DAY, NEXT_DAY = pd.Timestamp("2026-07-01", tz="UTC"),
        pd.Timestamp("2026-07-02", tz="UTC")
july_1 = pd.DataFrame({"n": [int(((df["time"] >= DAY) &
        (df["time"] < NEXT_DAY)).sum())]})
FIRST_DAY, SECOND_DAY = pd.Timestamp("2026-03-16",
        tz="UTC"), pd.Timestamp("2026-03-17", tz="UTC")
first_day = pd.DataFrame(
    {"n": [int(((df["time"] >= FIRST_DAY) & (df["time"] <
        SECOND_DAY)).sum())]}
)

# 5. The same day by computing a date for every row: same
        rows, but O(n) work every time.
july_1_by_date = pd.DataFrame(
    {"n": [int((df["time"].dt.strftime("%Y-%m-%d") ==
        "2026-07-01").sum())]}
)

# 6. A closed range on text that is too short misses the
        whole day (the text has a time on it).
sloppy = pd.DataFrame(
    {
        "n": [
            int(
                (
                    (df["timestamp_utc"] >= "2026-07-01") &
        (df["timestamp_utc"] <= "2026-07-01")
                ).sum()
            )
        ]
    }
)

# 7. Weekday: pandas counts Monday = 0, SQLite's %w counts
        Sunday = 0. Convert to compare.
first_weekday = (first.dayofweek + 1) % 7  # 2026-03-16 is
        a Monday: 0 in pandas, 1 in SQL
weekday = pd.DataFrame(
    {
        "first_weekday_sun0": [first_weekday],
        "weekday_rows": [int((df["time"].dt.dayofweek <
        5).sum())],  # Mon..Fri, UTC calendar
    }
)
```

```
# 8. When did the fan first start on 2026-04-01? (UTC;
        local time below.)
on = df[(df["supply_fan_status"] == 1) & (df["time"] >=
        "2026-04-01") & (df["time"] < "2026-04-02")]
fan_start = pd.DataFrame({"fan_start_utc":
        [on["time"].min().strftime(FMT)]})

# 9. The hottest reading, in UTC.
hot = df.loc[[df["outside_air_temp_f"].idxmax()]]
hottest = pd.DataFrame(
    {"time_utc": hot["time"].dt.strftime(FMT),
        "outside_air_temp_f": hot["outside_air_temp_f"]}
).reset_index(drop=True)

RESULTS = {
    "first3": first3,
    "span": span,
    "gap_check": gap_check,
    "july_1": july_1,
    "first_day": first_day,
    "july_1_by_date": july_1_by_date,
    "sloppy": sloppy,
    "weekday": weekday,
    "fan_start": fan_start,
    "hottest": hottest,
}

print("dtype of the parsed column:", df["time"].dtype)
print("\nfirst three readings, as dates and epoch
        seconds:")
print(first3.to_string(index=False))
print("\nfirst and last timestamp:")
print(span.to_string(index=False))
print("rows if every 5-minute step is present:",
        gap_check["expected_rows"][0])
print("rows actually there:                    ",
        gap_check["actual_rows"][0])
print("\nreadings on 2026-07-01 (half-open range):",
        july_1["n"][0])
print("readings on 2026-03-16 (data starts at 00:40):",
        first_day["n"][0])
print("same day via a computed date on every row:",
        july_1_by_date["n"][0])
print(
    "BETWEEN '2026-07-01' AND '2026-07-01' on text:",
    sloppy["n"][0],
    "(the day's rows all sort after it)",
)
print("\nweekday of the first row (0 = Sunday, as SQL
        counts):", weekday["first_weekday_sun0"][0])
```

```
print("Monday-to-Friday readings (UTC calendar):",
          weekday["weekday_rows"][0])
print(
    "\nthe hottest reading, UTC:",
    hottest["time_utc"][0],
    round(hottest["outside_air_temp_f"][0], 2),
)

# Local time is a job for pandas: it knows named zones and
          daylight saving. SQLite does not.
CHICAGO = "America/Chicago"
print("  ... and in", CHICAGO, ":",
          hot["time"].iloc[0].tz_convert(CHICAGO))
start_local = on["time"].min().tz_convert(CHICAGO)
print(
    "fan first started on 2026-04-01 at",
    fan_start["fan_start_utc"][0],
    "UTC =",
    start_local,
    "local",
)

# Naive vs aware: pandas refuses to compare them. That
          error is the friendly outcome.
naive = pd.to_datetime(df["timestamp_utc"].str.slice(0,
          19))  # drops the +00:00: no zone
try:
    _ = naive >= DAY
except TypeError as err:
    print("\nTypeError:", err)
```

day03_sql.py

```
"""Day 3, SQL: time. ISO text timestamps,
         datetime()/strftime()/unixepoch(), half-open
         ranges.

    uv run python lessons/course.py setup         # once:
         loads the CSVs into var/lessons.sqlite
    uv run python lessons/unit1-sqlite/day03_sql.py

SQLite has no date type. timestamp_utc is TEXT like '2026-
         03-16 00:40:00+00:00', and the date
functions parse that text row by row. SQLite does all the
         work; pandas only turns the result rows
into a table for printing.
Compare the printout with day03_pandas.py: the numbers
         should match.
"""

import os
import sqlite3
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DB = Path(os.environ.get("LESSONS_DB", ROOT / "var" /
         "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv run python
         lessons/course.py setup")
conn = sqlite3.connect(DB)


def q(sql, params=None):
    """Run one SELECT with its parameters and return a
         DataFrame."""
    return pd.read_sql_query(sql, conn, params=params or
         {})


# 1. Nothing to parse up front: the column is text, and the
         functions below read it directly.
TYPEOF_SQL = "SELECT typeof(timestamp_utc) AS stored_as
         FROM ahu_1 LIMIT 1"

# 2. The first three readings as dates and as Unix epoch
         seconds (seconds since 1970-01-01 UTC).
FIRST3_SQL = """
SELECT datetime(timestamp_utc)   AS time_utc,
       unixepoch(timestamp_utc)  AS epoch_s,
```

```
       outside_air_temp_f
FROM ahu_1
ORDER BY timestamp_utc
LIMIT 3
"""

# 3. First and last timestamp, and a gap check: 5-minute
        steps from first to last need N rows.
SPAN_SQL = """
SELECT datetime(min(timestamp_utc)) AS first_utc,
        datetime(max(timestamp_utc)) AS last_utc
FROM ahu_1
"""
GAP_CHECK_SQL = """
SELECT unixepoch(max(timestamp_utc)) -
        unixepoch(min(timestamp_utc))        AS span_s,
       (unixepoch(max(timestamp_utc)) -
        unixepoch(min(timestamp_utc))) / 300 + 1 AS
        expected_rows,
       count(*)
        AS actual_rows
FROM ahu_1
"""

# 4. A day, as a HALF-OPEN range: start <= time < end, so
        consecutive days never share a reading.
# The bounds are text in the column's own format, so SQLite
        compares the raw column to them and can
# seek in the PRIMARY KEY index: O(log n + k).
DAY_SQL = """
SELECT count(*) AS n FROM ahu_1 WHERE timestamp_utc >=
        :start AND timestamp_utc < :end
"""

# 5. The same day by computing a date for every row: same
        rows, but SQLite can't use the index.
BY_DATE_SQL = "SELECT count(*) AS n FROM ahu_1 WHERE
        date(timestamp_utc) = :day"

# 6. A closed range on text that is too short misses the
        whole day (the text has a time on it).
SLOPPY_SQL = """
SELECT count(*) AS n FROM ahu_1 WHERE timestamp_utc BETWEEN
        :start AND :end
"""

# 7. Weekday: SQLite's %w counts Sunday = 0 and returns
        TEXT; pandas counts Monday = 0.
WEEKDAY_SQL = """
SELECT CAST(strftime('%w', min(timestamp_utc)) AS INTEGER)
        AS first_weekday_sun0,
```

```
       sum(CAST(strftime('%w', timestamp_utc) AS INTEGER)
         BETWEEN 1 AND 5) AS weekday_rows
FROM ahu_1
"""

# 8. When did the fan first start on 2026-04-01? (UTC;
        local time below.)
FAN_START_SQL = """
SELECT datetime(min(timestamp_utc)) AS fan_start_utc
FROM ahu_1
WHERE supply_fan_status = 1 AND timestamp_utc >= :start AND
        timestamp_utc < :end
"""

# 9. The hottest reading, in UTC.
HOTTEST_SQL = """
SELECT datetime(timestamp_utc) AS time_utc,
        outside_air_temp_f
FROM ahu_1
ORDER BY outside_air_temp_f DESC, timestamp_utc
LIMIT 1
"""

# Text in the same format as the column. pandas builds the
        same strings with str(Timestamp).
JULY_1 = {"start": "2026-07-01 00:00:00+00:00", "end":
        "2026-07-02 00:00:00+00:00"}
MARCH_16 = {"start": "2026-03-16 00:00:00+00:00", "end":
        "2026-03-17 00:00:00+00:00"}
APRIL_1 = {"start": "2026-04-01 00:00:00+00:00", "end":
        "2026-04-02 00:00:00+00:00"}

stored_as = q(TYPEOF_SQL)["stored_as"][0]
first3 = q(FIRST3_SQL)
span = q(SPAN_SQL)
gap_check = q(GAP_CHECK_SQL)
july_1 = q(DAY_SQL, JULY_1)
first_day = q(DAY_SQL, MARCH_16)
july_1_by_date = q(BY_DATE_SQL, {"day": "2026-07-01"})
sloppy = q(SLOPPY_SQL, {"start": "2026-07-01", "end":
        "2026-07-01"})
weekday = q(WEEKDAY_SQL)
fan_start = q(FAN_START_SQL, APRIL_1)
hottest = q(HOTTEST_SQL)

# Which plan did SQLite pick? SEARCH ... USING ... INDEX is
        a seek; SCAN reads every row.
plan_range = conn.execute("EXPLAIN QUERY PLAN " + DAY_SQL,
        JULY_1).fetchall()[0][-1]
plan_date = conn.execute("EXPLAIN QUERY PLAN " +
        BY_DATE_SQL, {"day": "2026-07-01"}).fetchall()[0][
```

```
    -1
]

RESULTS = {
    "first3": first3,
    "span": span,
    "gap_check": gap_check,
    "july_1": july_1,
    "first_day": first_day,
    "july_1_by_date": july_1_by_date,
    "sloppy": sloppy,
    "weekday": weekday,
    "fan_start": fan_start,
    "hottest": hottest,
}

print("timestamp_utc is stored as:", stored_as)
print("\nfirst three readings, as dates and epoch
        seconds:")
print(first3.to_string(index=False))
print("\nfirst and last timestamp:")
print(span.to_string(index=False))
print("rows if every 5-minute step is present:",
        gap_check["expected_rows"][0])
print("rows actually there:                    ",
        gap_check["actual_rows"][0])
print("\nreadings on 2026-07-01 (half-open range):",
        july_1["n"][0])
print("readings on 2026-03-16 (data starts at 00:40):",
        first_day["n"][0])
print("same day via a computed date on every row:",
        july_1_by_date["n"][0])
print(
    "BETWEEN '2026-07-01' AND '2026-07-01' on text:",
    sloppy["n"][0],
    "(the day's rows all sort after it)",
)
print("  plan, range  :", plan_range)
print("  plan, date() :", plan_date)
print("\nweekday of the first row (0 = Sunday, as SQL
        counts):", weekday["first_weekday_sun0"][0])
print("Monday-to-Friday readings (UTC calendar):",
        weekday["weekday_rows"][0])
print(
    "\nthe hottest reading, UTC:",
    hottest["time_utc"][0],
    round(hottest["outside_air_temp_f"][0], 2),
)

# SQLite has no named zones, only 'localtime': the zone of
        whatever machine runs the query.
```

```
local = conn.execute(
    "SELECT datetime(:ts, 'localtime')", {"ts":
         hottest["time_utc"][0]}
).fetchone()[0]
print("  ... 'localtime' on THIS machine:", local, "
         (changes with the machine's zone)")
print("fan first started on 2026-04-01 at",
         fan_start["fan_start_utc"][0], "UTC")
```

Day 4 — Summary stats: COUNT, MIN, MAX, AVG, SUM

Goal: summarize a column in one row, in pandas and in SQL, and turn a status point into run hours.

Read

Aggregate functions fold many rows into one value. pandas has a method for each:

SQL	pandas (on a column v)	Notes
`count(*)`	`len(df)`	rows
`count(col)`	`v.count()`	rows where the value is not NULL / NaN
`min(col)`, `max(col)`	`v.min()`, `v.max()`	ignore NULLs / NaN
`avg(col)`	`v.mean()`	mean of the non-NULL values
`sum(col)`	`v.sum()`	total (SQL: NULL for no rows; pandas: 0)
`round(x, n)`	`round(x, n)`	for reading, not storage. **Never round before comparing**

Without `GROUP BY` (tomorrow), an aggregate query returns exactly one row.

NULLs. A missing reading is `NULL` in SQL and `NaN` in pandas. Both skip it in `min`, `max`, `avg` and `sum`, and both agree that `count(col)` counts only the values that are there, while `count(*)` counts rows. So the mean of `1, <missing>, 3` is 2.0, not 4/3. The Building 50 columns have no gaps, so the scripts build a three-row example to show it (a throwaway in-memory SQLite table in the SQL script, a `pd.Series` in pandas). In real data a flat-

lined or missing sensor often shows up as NULLs, or as a suspicious constant like `0` or `-999`; only the first is skipped for you.

Run hours. A status point is `1` when running and 0 when off. With one reading every 5 minutes, `sum(status) × 5 / 60` is the **run hours**. Use `60.0`, not `60`, in SQL: with two integers SQLite divides as integers and drops the remainder (`7 * 5 / 60` is `0`). Python 3's `/` is always true division; its `//` is the one that floors. Different engines, different gotchas.

How each engine does it. Both are one pass over the rows, O(n). SQLite computes every aggregate in the `SELECT` list in the *same* pass, while it streams the rows off the table, and returns a single row. pandas works on columns in memory: each of `min`, `max`, `mean` is its own fast, vectorized pass (a compiled C loop, not a Python `for`), and the rows had to be loaded into Python first. So SQL moves one row of results; pandas moves all the input. Nothing here is faster in an *algorithmic* sense. The aggregate is O(n) either way, and no index helps with `avg()`. What differs is how much data crosses from the database into your program.

Run

```
uv run python lessons/course.py run 4
```

That runs `day04_pandas.py`, then `day04_sql.py`. Both print:

```
AHU_1 outside air temperature, whole dataset:
    n  n_values  min_f  max_f  avg_f
35536     35536  17.83  98.86 61.276

the same for 2026-07-01:
 min_f  max_f  avg_f
  79.9   97.1   88.3

supply fan run hours (readings with the fan on x 5 min /
60):
equipment_id  run_hours  fraction_running
       AHU_1   1574.417             0.532
       AHU_2   1499.583             0.506

AHU_1 duct static and its setpoint, averaged over every
hour (inH2O):
 static_inwc  static_setpoint_inwc
       0.783                1.366
```

How each step maps across:

Step	`day04_sql.py`	`day04_pandas.py`
four stats in one pass	`SELECT count(*), min(c), max(c), avg(c) FROM ahu_1`	`v.count()`, `v.min()`, `v.max()`, `v.mean()`
one day	`WHERE timestamp_utc >= :start AND timestamp_utc < :end`	mask on the text, then the same methods
run hours	`sum(supply_fan_status) * 5 / 60.0`	`s.sum() * 5 / 60`
both AHUs	`SELECT 'AHU_1' ... UNION ALL SELECT 'AHU_2' ...`	a dict of DataFrames, one per CSV
fraction of time on	`avg(supply_fan_status)`	`s.mean()`

`UNION ALL` stacks the rows of two queries into one result; it is how the SQL script brings AHU_2 in next to AHU_1.

The dataset covers about 2,961 hours (35,536 readings × 5 minutes), so AHU_1's fan ran 1,574 of them, 53.2% of the time. (The average of a 0/1 column is the fraction of rows that are 1.)

All numbers are printed rounded, but compared unrounded: the checks behind these lessons compare the raw values to 1e-9, and rounding before you compare turns “equal” into “equal to 3 decimals”.

Try it

1. What are AHU_2’s average outside air temperature and its duct static/setpoint averages? How do they compare with AHU_1’s? (Add AHU_2 to the SQL with a second `FROM`, or `UNION ALL`.)
2. How many hours was AHU_1 scheduled occupied? Occupancy is column `occ_c` (1 = occupied). Compare with the fan’s run hours.

Answers

1. AHU_2: outside air 63.42 °F (AHU_1: 61.28), duct static 0.785 inH2O against a setpoint average of 0.775. The two outside-air averages differ by 2 °F even though both AHUs sit in the same building: they are different sensors in different places, so even the “same” quantity needs checking before you compare it. And AHU_2’s average static is at its setpoint, where AHU_1’s static is far below its (1.366). That gap is today’s hint at a problem on AHU_1. But it includes all the hours the fan was off, when static is near zero. Day 8 asks the fair question: what is static *while the fan runs*?

```sql
SELECT avg(outside_air_temp_f), avg(da_p_inwc),
       avg(da_p_setpoint_inwc) FROM ahu_2
```

2. 1,518.0 occupied hours (`sum(occ_c) * 5 / 60.0`) against 1,574.4 fan hours: the fan runs 56 hours more than the schedule says the building is occupied, on top of any hours it runs for warm-up or night purge. (AHU_2: 1,519.25 occupied hours, 1,499.6 fan hours.) Whether that is waste or sensible depends on the sequence of operations, and it is the kind of question Week 3’s gating makes precise.

Why it matters

Averages over everything hide what matters. A static pressure reading near 0 because the fan is off pulls the average down just like a real problem does. Most of fault detection is asking "the average **of what, when**?" That is filtering (Days 8–9) and gating (Week 3). Run hours are the first example of a number a building owner actually reads: hours the fan ran, hours a fault was active, hours of energy at a given load.

day04_pandas.py

```
"""Day 4, pandas: summary statistics (count, min, max,
        mean, sum) and fan run hours.

    uv run python lessons/unit1-sqlite/day04_pandas.py

One column in, one number out. The status point
        supply_fan_status is 1 when the fan runs and 0
        when
it is off; with a reading every 5 minutes, sum(status) * 5
        / 60 is the run hours.
Compare the printout with day04_sql.py: the numbers should
        match.
"""

from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"

# 1. Load the columns we summarize, for both AHUs. Same
        column names in both exports.
COLUMNS = [
    "timestamp_utc",
    "outside_air_temp_f",
    "da_p_inwc",
    "da_p_setpoint_inwc",
    "supply_fan_status",
]
frames = {
    e: pd.read_csv(DATA / e / "history_wide.csv",
        usecols=COLUMNS) for e in ("AHU_1", "AHU_2")
}
ahu_1 = frames["AHU_1"]

# 2. AHU_1 outside air temperature, whole dataset: four
        statistics, unrounded.
v = ahu_1["outside_air_temp_f"]
oat_summary = pd.DataFrame(
    {
        "n": [len(v)],
        "n_values": [v.count()],
        "min_f": [v.min()],
        "max_f": [v.max()],
        "avg_f": [v.mean()],
    }
```

```
)

# 3. The same for one day: a mask first (Day 2's half-open
        range on the ISO text), then stats.
START, END = "2026-07-01 00:00:00+00:00", "2026-07-02
        00:00:00+00:00"
day = ahu_1[(ahu_1["timestamp_utc"] >= START) &
        (ahu_1["timestamp_utc"] < END)][
    "outside_air_temp_f"
]
oat_july_1 = pd.DataFrame({"min_f": [day.min()], "max_f":
        [day.max()], "avg_f": [day.mean()]})

# 4. Run hours of the supply fan, per AHU: sum of a 0/1
        column x 5 minutes / 60.
run_hours = pd.DataFrame(
    {
        "equipment_id": list(frames),
        "run_hours": [f["supply_fan_status"].sum() * 5 / 60
         for f in frames.values()],
        "fraction_running": [f["supply_fan_status"].mean()
         for f in frames.values()],
    }
)

# 5. Averages that mislead: static pressure averaged over
        ALL hours, fan on and off.
averages = pd.DataFrame(
    {
        "static_inwc": [ahu_1["da_p_inwc"].mean()],
        "static_setpoint_inwc":
         [ahu_1["da_p_setpoint_inwc"].mean()],
    }
)

# 6. NULLs: pandas stores a missing value as NaN and skips
        it. (The real columns have none.)
t = pd.Series([1.0, np.nan, 3.0])
null_demo = pd.DataFrame({"n_rows": [len(t)], "n_values":
        [t.count()], "mean": [t.mean()]})

# 7. Integer division: Python 3's / is always true
        division; // is the one that drops the rest.
division = pd.DataFrame({"floor_div": [7 * 5 // 60],
        "true_div": [7 * 5 / 60]})

RESULTS = {
    "oat_summary": oat_summary,
    "oat_july_1": oat_july_1,
    "run_hours": run_hours,
    "averages": averages,
```

```
    "null_demo": null_demo,
    "division": division,
}

print("AHU_1 outside air temperature, whole dataset:")
print(oat_summary.round({"min_f": 2, "max_f": 2, "avg_f":
        3}).to_string(index=False))
print("\nthe same for 2026-07-01:")
print(oat_july_1.round(1).to_string(index=False))
print("\nsupply fan run hours (readings with the fan on x 5
        min / 60):")
print(run_hours.round(3).to_string(index=False))
print("\nAHU_1 duct static and its setpoint, averaged over
        every hour (inH2O):")
print(averages.round(3).to_string(index=False))
print("\nthree values, one missing:")
print(null_demo.round(3).to_string(index=False))
print("\n7 * 5 / 60 two ways (floor, true):",
        division.iloc[0].round(4).tolist())
```

day04_sql.py

```
"""Day 4, SQL: COUNT, MIN, MAX, AVG, SUM, NULL handling and
        fan run hours.

    uv run python lessons/course.py setup         # once:
        loads the CSVs into var/lessons.sqlite
    uv run python lessons/unit1-sqlite/day04_sql.py

An aggregate folds many rows into one value. All the
        aggregates in one SELECT are computed in the
same pass over the rows, and one row comes back. SQLite
        does all the work; pandas only turns the
result rows into a table for printing.
Compare the printout with day04_pandas.py: the numbers
        should match.
"""

import os
import sqlite3
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DB = Path(os.environ.get("LESSONS_DB", ROOT / "var" /
        "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv run python
        lessons/course.py setup")
conn = sqlite3.connect(DB)


def q(sql, params=None, db=conn):
    """Run one SELECT with its parameters and return a
        DataFrame."""
    return pd.read_sql_query(sql, db, params=params or {})


# 1. Nothing to load: the tables `ahu_1` and `ahu_2` hold
        the same rows as the CSVs.

# 2. AHU_1 outside air temperature, whole dataset. count(*)
        counts rows; count(col) skips NULLs.
OAT_SUMMARY_SQL = """
SELECT count(*)                  AS n,
       count(outside_air_temp_f) AS n_values,
       min(outside_air_temp_f)   AS min_f,
       max(outside_air_temp_f)   AS max_f,
       avg(outside_air_temp_f)   AS avg_f
```

```
FROM ahu_1
"""

# 3. The same for one day: a half-open range in the WHERE
        (Day 2), then the same aggregates.
OAT_DAY_SQL = """
SELECT min(outside_air_temp_f) AS min_f,
       max(outside_air_temp_f) AS max_f,
       avg(outside_air_temp_f) AS avg_f
FROM ahu_1
WHERE timestamp_utc >= :start AND timestamp_utc < :end
"""

# 4. Run hours of the supply fan, per AHU: sum of a 0/1
        column x 5 minutes / 60.0.
# Use 60.0, not 60: with two integers SQL divides as
        integers and drops the remainder.
RUN_HOURS_SQL = """
SELECT 'AHU_1' AS equipment_id,
       sum(supply_fan_status) * 5 / 60.0 AS run_hours,
       avg(supply_fan_status)            AS
         fraction_running
FROM ahu_1
UNION ALL
SELECT 'AHU_2', sum(supply_fan_status) * 5 / 60.0,
        avg(supply_fan_status)
FROM ahu_2
"""

# 5. Averages that mislead: static pressure averaged over
        ALL hours, fan on and off.
AVERAGES_SQL = """
SELECT avg(da_p_inwc)          AS static_inwc,
       avg(da_p_setpoint_inwc) AS static_setpoint_inwc
FROM ahu_1
"""

# 6. NULLs: a three-row table in a throwaway in-memory
        database. Aggregates skip the NULL.
mem = sqlite3.connect(":memory:")
mem.execute("CREATE TABLE t (value REAL)")
mem.executemany("INSERT INTO t VALUES (?)", [(1.0,),
        (None,), (3.0,)])
NULL_DEMO_SQL = "SELECT count(*) AS n_rows, count(value) AS
        n_values, avg(value) AS mean FROM t"

# 7. Integer division: 7 * 5 / 60 is 0 in SQL (integers), 7
        * 5 / 60.0 is 0.5833.
DIVISION_SQL = "SELECT 7 * 5 / 60 AS floor_div, 7 * 5 /
        60.0 AS true_div"
```

```
START, END = "2026-07-01 00:00:00+00:00", "2026-07-02
        00:00:00+00:00"
oat_summary = q(OAT_SUMMARY_SQL)
oat_july_1 = q(OAT_DAY_SQL, {"start": START, "end": END})
run_hours = q(RUN_HOURS_SQL)
averages = q(AVERAGES_SQL)
null_demo = q(NULL_DEMO_SQL, db=mem)
division = q(DIVISION_SQL)

RESULTS = {
    "oat_summary": oat_summary,
    "oat_july_1": oat_july_1,
    "run_hours": run_hours,
    "averages": averages,
    "null_demo": null_demo,
    "division": division,
}

print("AHU_1 outside air temperature, whole dataset:")
print(oat_summary.round({"min_f": 2, "max_f": 2, "avg_f":
        3}).to_string(index=False))
print("\nthe same for 2026-07-01:")
print(oat_july_1.round(1).to_string(index=False))
print("\nsupply fan run hours (readings with the fan on x 5
        min / 60):")
print(run_hours.round(3).to_string(index=False))
print("\nAHU_1 duct static and its setpoint, averaged over
        every hour (inH2O):")
print(averages.round(3).to_string(index=False))
print("\nthree values, one missing:")
print(null_demo.round(3).to_string(index=False))
print("\n7 * 5 / 60 two ways (floor, true):",
        division.iloc[0].round(4).tolist())
```

Day 5 — GROUP BY and HAVING: per day, per month, and the Week 1 challenge

Goal: one summary row per group, keep only the groups you care about, in pandas and in SQL, and answer a three-measure question per month.

Read

`GROUP BY` splits rows into groups. Each aggregate is then computed once per group:

```
SELECT group_columns, aggregates
FROM ...
WHERE   row filter           -- before grouping
GROUP BY group_columns
HAVING  group filter          -- after grouping, can use
aggregates
ORDER BY ...
```

`WHERE` throws away rows before they are grouped. `HAVING` throws away whole groups after. You can't put `avg(...)` in a `WHERE`, because no group exists yet at that point.

You can group by an expression, such as the date: `GROUP BY date(timestamp_utc)` gives one row per day, and `strftime('%Y-%m', timestamp_utc)` one row per month (Day 3's date functions). SQLite lets you use a column alias (`AS day`) in `GROUP BY` and `ORDER BY`. The pandas version of each clause:

SQL	pandas
`WHERE cond`	`df[mask]` before the groupby
`GROUP BY key`	`df.groupby(key)` (key can be a column or a computed Series, like a formatted date)
`avg(c)`, `count(*)`, `sum(c)`	`.agg(avg_f=("c", "mean"), n=("c", "size"), ...)`

SQL	pandas
`HAVING count(*) < 288`	filter **after** the groupby: `grouped[grouped.n < 288]`
`ORDER BY x DESC LIMIT 1`	`.sort_values("x", ascending=False).head(1)`
`sum(da_p_inwc > 1.5)`	`(df.da_p_inwc > 1.5).sum()`

So `HAVING` is not a special pandas feature. It is just an ordinary filter on the small table `groupby` gives you. `WHERE` and `HAVING` differ in SQL only because of clause order; in pandas the order of your statements says it.

Counting with a comparison. In SQLite a comparison such as `da_p_inwc > 1.5` is `1` when true and `0` when false, so `sum(da_p_inwc > 1.5)` counts the rows where it holds, in the same pass as any other aggregate. pandas does the same with a boolean column (`True` is `1` when you `sum`). That makes "count the readings that meet a condition, per group" a one-liner, and it is the shape most fault-detection summaries take.

How each engine groups. pandas **hashes**: one pass over the rows, each row dropped into a bucket for its key, O(n). SQLite either **sorts** the rows by the key (into a temporary B-tree) and sums runs of equal keys, O(n log n), or, if the rows already come out in key order, streams them without a sort. `timestamp_utc` is the primary key, so rows come out in time order, but a *day* is computed from it and SQLite doesn't know the computed values are in order, so `GROUP BY date(timestamp_utc)` needs the temporary B-tree. The SQL script prints the plan: `SCAN ahu_1 | USE TEMP B-TREE FOR GROUP BY`. At 35,536 rows this costs nothing you can notice.

Run

```
uv run python lessons/course.py run 5
```

That runs `day05_pandas.py`, then `day05_sql.py`. Each step maps like this:

Step	day05_sql.py	day05_pandas.py
day and month labels	`date(timestamp_utc)`, `strftime('%Y-%m', timestamp_utc)`	`t.dt.strftime("%Y-%m-%d")`, `"%Y-%m"`
monthly average	`GROUP BY month` + `avg(...)`	`groupby("month").agg(avg_f=(..., "mean"))`
hottest day	`GROUP BY day ORDER BY avg_f DESC LIMIT 1`	`groupby` then `sort_values(...).head(1)`
short days	`HAVING count(*) < :full_day`	`counts[counts.n < 288]`
hot readings per day	`WHERE ... > :hot_f GROUP BY day`	mask first, then `groupby`
daily run hours	`sum(supply_fan_status) * 5 / 60.0`	`s.sum() * 5 / 60` in an `agg`
both AHUs	a temp view of `ahu_1 UNION ALL ahu_2`	`pd.concat` of the two CSVs

Both print:

```
AHU_1 monthly average outside air temperature (F):
  month  avg_f
2026-03   41.5
2026-04   53.0
2026-05   59.8
2026-06   71.3
2026-07   80.0

the hottest day on average:
       day  avg_f
2026-07-01  88.30

days with fewer than 288 readings:
       day   n
2026-03-16 280
2026-07-17 120

days that ever passed 90 F: 10 of 124
days the fan ran all 24 hours: 13 of 124
```

1 July (88.30) edges out 2 July (88.25) by 0.04 °F, so the two look tied at one decimal. The sort used the unrounded averages. Round for display only.

The two short days are the first (2026-03-16, starts at 00:40, 280 readings) and the last (2026-07-17, ends at 09:55, 120 readings). There are no gaps in between. Real historians usually aren't this tidy, so checking is always worth it.

`WHERE` vs `HAVING`: the hot-readings query filters rows *before* grouping, so days with no reading above 90 °F have no rows to group and vanish from the result. That is why it returns 10 of the 124 days, not 124 rows with a zero. A `HAVING` can only remove groups that exist.

Thirteen days had the fan running all 24 hours, mostly in the hot spells of late June and July. Is that normal for an office AHU? That is a question for the building's operators, and it is the kind of finding retro-commissioning starts from.

Try it

1. AHU_1's fan run hours per month: which month had the most?
2. Which months had any readings with duct static above 1.5 inH2O? (Use the Week 1 challenge's output.)
3. Show only the months where AHU_1's fan ran more than 400 hours. Where does that condition go, `WHERE` or `HAVING`?

Answers

1. June, with 411.6 hours (March 182.7, April 341.9, May 331.7, July 306.6; March and July are partial months).

2. Only June (4,721 readings on AHU_1) and July (3,615). In March, April and May AHU_1's static never went above 1.5. In pandas: filter the challenge table with `challenge[challenge.static_above_1_5 > 0]`.

3. `HAVING`, because it is a condition on an aggregate (`sum(...)`) of the group:

```sql
SELECT strftime('%Y-%m', timestamp_utc) AS month,
       sum(supply_fan_status) * 5 / 60.0 AS run_hours
FROM ahu_1
GROUP BY month
HAVING sum(supply_fan_status) * 5 / 60.0 > 400
```

Only `2026-06` comes back. In pandas it is `monthly[monthly.run_hours > 400]`, after the groupby.

Week 1 challenge

For each AHU and each month, report **the average outside air temperature, the fan run hours, and the number of readings where duct static was above 1.5 inH2O**. One result, ten rows (two AHUs × five months).

This is step 7 in both scripts. Try writing it yourself first, in each tool, before you read the scripts' versions. What to notice:

- In the old long layout the three measures lived in three different series, and one query could not easily combine them. In the wide table all three are columns of the same row, so one `GROUP BY` and three aggregates answer the question: `avg(outside_air_temp_f)`, `sum(supply_fan_status) * 5 / 60.0`, `sum(da_p_inwc > 1.5)`.
- Grouping by two keys, `equipment_id` and `month`, gives one row per pair. `equipment_id` is not a column of either table, so both scripts add it when they stack the AHUs (`SELECT 'AHU_1' AS equipment_id ... UNION ALL ...` in SQL, `insert(0, "equipment_id", ...)` and `concat` in pandas).

Both scripts print the same table; the AHU_1 rows are:

```
equipment_id   month  avg_oat_f  fan_run_hours
static_above_1_5
       AHU_1 2026-03       41.5          182.7
0
       AHU_1 2026-04       53.0          341.9
0
       AHU_1 2026-05       59.8          331.7
0
       AHU_1 2026-06       71.3          411.6
4721
       AHU_1 2026-07       80.0          306.6
3615
```

Static above 1.5 shows up only once it is warm, in June and July, for both AHUs. Next week's questions are about *why*, and whether that is a problem: AHU_1's average setpoint is 1.37

inH2O, so a reading above 1.5 might be the system doing exactly what it was asked.

Why it matters

A summary table per day or per month is the first thing a building owner will ask for, and `GROUP BY` plus a filter on the groups is how every such table is made. The group key does a lot of quiet work: group by a date you computed in UTC and "a day" starts at 7 pm local time in summer, which is a mistake you will not see in the numbers. Before judging a reading you also have to say *which* readings count (when the fan is on, when the building is occupied).

day05_pandas.py

```python
"""Day 5, pandas: groupby per day and per month, filtering
        groups, and the Week 1 challenge.

    uv run python lessons/unit1-sqlite/day05_pandas.py

groupby splits rows into groups and an aggregate runs once
        per group. SQL's WHERE is a mask before
the groupby; SQL's HAVING is an ordinary filter on the
        small table groupby gives back.
Compare the printout with day05_sql.py: the numbers should
        match.
"""

from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"
FULL_DAY = 288  # 24 h x 12 readings an hour

# 1. Load both AHUs, the columns we group. day and month
        are the text of the timestamp's date parts.
COLUMNS = ["timestamp_utc", "outside_air_temp_f",
        "da_p_inwc", "supply_fan_status"]
frames = []
for equip in ("AHU_1", "AHU_2"):
    f = pd.read_csv(DATA / equip / "history_wide.csv",
        usecols=COLUMNS)
    f.insert(0, "equipment_id", equip)
    frames.append(f)
h = pd.concat(frames, ignore_index=True)
time = pd.to_datetime(h["timestamp_utc"], utc=True)
h["day"] = time.dt.strftime("%Y-%m-%d")
h["month"] = time.dt.strftime("%Y-%m")
ahu_1 = h[h["equipment_id"] == "AHU_1"]

# 2. Monthly average outside air temperature: groupby a
        column, agg for the statistic.
monthly_avg = ahu_1.groupby("month").agg(avg_f=
        ("outside_air_temp_f", "mean")).reset_index()

# 3. Readings per day, then the hottest day on average
        (sort the small result, keep the top row).
daily_counts = ahu_1.groupby("day").agg(n=
        ("outside_air_temp_f", "size")).reset_index()
daily_avg = ahu_1.groupby("day").agg(avg_f=
        ("outside_air_temp_f", "mean")).reset_index()
```

```
hottest_day = daily_avg.sort_values("avg_f",
          ascending=False).head(1).reset_index(drop=True)

# 4. HAVING: days without a full 288 readings. A filter
          AFTER the groupby.
short_days = daily_counts[daily_counts["n"] <
          FULL_DAY].reset_index(drop=True)

# 5. WHERE before GROUP BY: readings above 90 F per day.
          Days with none have no rows to group.
hot_counts = (
    ahu_1[ahu_1["outside_air_temp_f"] > 90]
    .groupby("day")
    .agg(n_hot=("outside_air_temp_f", "size"))
    .reset_index()
)

# 6. Daily fan run hours; the three longest days, and how
          many days ran all 24 hours.
daily_hours = (
    ahu_1.groupby("day")
    .agg(run_hours=("supply_fan_status", lambda s: s.sum()
          * 5 / 60))
    .reset_index()
)
longest_days = (
    daily_hours.sort_values(["run_hours", "day"],
          ascending=[False, True])
    .head(3)
    .reset_index(drop=True)
)
full_run_days = daily_hours[daily_hours["run_hours"] >=
          24].reset_index(drop=True)

# 7. Week 1 challenge: per AHU and month, the average
          outside air temperature, the fan run hours,
# and the number of readings with duct static above 1.5
          inH2O. One groupby on two keys.
h["static_high"] = h["da_p_inwc"] > 1.5
challenge = (
    h.groupby(["equipment_id", "month"])
    .agg(
        avg_oat_f=("outside_air_temp_f", "mean"),
        fan_run_hours=("supply_fan_status", lambda s:
          s.sum() * 5 / 60),
        static_above_1_5=("static_high", "sum"),
    )
    .reset_index()
)
challenge["static_above_1_5"] =
          challenge["static_above_1_5"].astype(int)
```

```
RESULTS = {
    "monthly_avg": monthly_avg,
    "daily_counts": daily_counts,
    "hottest_day": hottest_day,
    "short_days": short_days,
    "hot_counts": hot_counts,
    "longest_days": longest_days,
    "full_run_days": full_run_days,
    "challenge": challenge,
}

print("AHU_1 monthly average outside air temperature (F):")
print(monthly_avg.round(1).to_string(index=False))
print("\nthe hottest day on average:")
print(hottest_day.round(2).to_string(index=False))
print("\ndays with fewer than 288 readings:")
print(short_days.to_string(index=False))
print(f"\ndays that ever passed 90 F: {len(hot_counts)} of
        {len(daily_counts)}")
print("\nthe three longest fan run days (hours):")
print(longest_days.round(1).to_string(index=False))
print(f"days the fan ran all 24 hours: {len(full_run_days)}
        of {len(daily_hours)}")
print("\nWeek 1 challenge, per AHU and month:")
print(challenge.round({"avg_oat_f": 1, "fan_run_hours":
        1}).to_string(index=False))
```

day05_sql.py

```
"""Day 5, SQL: GROUP BY and HAVING per day and per month,
        and the Week 1 challenge.

    uv run python lessons/course.py setup         # once:
        loads the CSVs into var/lessons.sqlite
    uv run python lessons/unit1-sqlite/day05_sql.py

GROUP BY splits rows into groups and each aggregate runs
        once per group. WHERE throws rows away
before they are grouped; HAVING throws whole groups away
        after, so it can use aggregates.
SQLite does all the work; pandas only turns the result rows
        into a table for printing.
Compare the printout with day05_pandas.py: the numbers
        should match.
"""

import os
import sqlite3
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DB = Path(os.environ.get("LESSONS_DB", ROOT / "var" /
        "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv run python
        lessons/course.py setup")
conn = sqlite3.connect(DB)


def q(sql, params=None):
    """Run one SELECT with its parameters and return a
        DataFrame."""
    return pd.read_sql_query(sql, conn, params=params or
        {})


# 1. Both AHUs, the columns we group, in a temp view. day
        and month are computed by SQLite's date
# functions from the timestamp text. (A view stores the
        query, not the rows.)
conn.execute("""
CREATE TEMP VIEW h AS
SELECT 'AHU_1' AS equipment_id, timestamp_utc,
        outside_air_temp_f, da_p_inwc, supply_fan_status,
       date(timestamp_utc) AS day, strftime('%Y-%m',
        timestamp_utc) AS month
```

```
FROM ahu_1
UNION ALL
SELECT 'AHU_2', timestamp_utc, outside_air_temp_f,
         da_p_inwc, supply_fan_status,
       date(timestamp_utc), strftime('%Y-%m',
         timestamp_utc)
FROM ahu_2
""")

# 2. Monthly average outside air temperature: GROUP BY an
         expression, aggregate per group.
MONTHLY_SQL = """
SELECT month, avg(outside_air_temp_f) AS avg_f
FROM h WHERE equipment_id = 'AHU_1'
GROUP BY month
ORDER BY month
"""

# 3. Readings per day, then the hottest day on average
         (sort the small result, keep the top row).
DAILY_COUNTS_SQL = """
SELECT day, count(*) AS n FROM h WHERE equipment_id =
         'AHU_1' GROUP BY day ORDER BY day
"""
HOTTEST_DAY_SQL = """
SELECT day, avg(outside_air_temp_f) AS avg_f
FROM h WHERE equipment_id = 'AHU_1'
GROUP BY day
ORDER BY avg_f DESC
LIMIT 1
"""

# 4. HAVING: days without a full 288 readings. The
         condition is on count(*), so it can't be WHERE.
SHORT_DAYS_SQL = """
SELECT day, count(*) AS n
FROM h WHERE equipment_id = 'AHU_1'
GROUP BY day
HAVING count(*) < :full_day
ORDER BY day
"""

# 5. WHERE before GROUP BY: readings above 90 F per day.
         Days with none have no rows to group.
HOT_COUNTS_SQL = """
SELECT day, count(*) AS n_hot
FROM h
WHERE equipment_id = 'AHU_1' AND outside_air_temp_f >
         :hot_f   -- row filter, before grouping
GROUP BY day
```

```
ORDER BY day
"""

# 6. Daily fan run hours; the three longest days, and the
#    days that ran all 24 hours.
LONGEST_DAYS_SQL = """
SELECT day, sum(supply_fan_status) * 5 / 60.0 AS run_hours
FROM h WHERE equipment_id = 'AHU_1'
GROUP BY day
ORDER BY run_hours DESC, day
LIMIT 3
"""
FULL_RUN_DAYS_SQL = """
SELECT day, sum(supply_fan_status) * 5 / 60.0 AS run_hours
FROM h WHERE equipment_id = 'AHU_1'
GROUP BY day
HAVING sum(supply_fan_status) * 5 / 60.0 >= 24
ORDER BY day
"""

# 7. Week 1 challenge: per AHU and month, the average
#    outside air temperature, the fan run hours,
# and the number of readings with duct static above 1.5
#    inH2O. One query: a comparison is 0 or 1,
# so sum() counts the rows where it is true.
CHALLENGE_SQL = """
SELECT equipment_id, month,
       avg(outside_air_temp_f)           AS avg_oat_f,
       sum(supply_fan_status) * 5 / 60.0 AS fan_run_hours,
       sum(da_p_inwc > 1.5)              AS
         static_above_1_5
FROM h
GROUP BY equipment_id, month
ORDER BY equipment_id, month
"""

monthly_avg = q(MONTHLY_SQL)
daily_counts = q(DAILY_COUNTS_SQL)
hottest_day = q(HOTTEST_DAY_SQL)
short_days = q(SHORT_DAYS_SQL, {"full_day": 288})
hot_counts = q(HOT_COUNTS_SQL, {"hot_f": 90})
longest_days = q(LONGEST_DAYS_SQL)
full_run_days = q(FULL_RUN_DAYS_SQL)
challenge = q(CHALLENGE_SQL)
n_days = len(daily_counts)

# How does SQLite group? Grouping by an expression of the
#    key needs a temporary sorted structure.
PLAN_SQL = "SELECT date(timestamp_utc) AS day,
         avg(outside_air_temp_f) FROM ahu_1 GROUP BY day"
```

```
plan = " | ".join(r[-1] for r in conn.execute("EXPLAIN
         QUERY PLAN " + PLAN_SQL))

RESULTS = {
    "monthly_avg": monthly_avg,
    "daily_counts": daily_counts,
    "hottest_day": hottest_day,
    "short_days": short_days,
    "hot_counts": hot_counts,
    "longest_days": longest_days,
    "full_run_days": full_run_days,
    "challenge": challenge,
}

print("AHU_1 monthly average outside air temperature (F):")
print(monthly_avg.round(1).to_string(index=False))
print("\nthe hottest day on average:")
print(hottest_day.round(2).to_string(index=False))
print("\ndays with fewer than 288 readings:")
print(short_days.to_string(index=False))
print(f"\ndays that ever passed 90 F: {len(hot_counts)} of
        {n_days}")
print("\nthe three longest fan run days (hours):")
print(longest_days.round(1).to_string(index=False))
print(f"days the fan ran all 24 hours: {len(full_run_days)}
        of {n_days}")
print("\nWeek 1 challenge, per AHU and month:")
print(challenge.round({"avg_oat_f": 1, "fan_run_hours":
        1}).to_string(index=False))
print("\nquery plan for GROUP BY day:", plan)
```

Reading — Week 1

The lessons first, then these for exact behavior.

- Python sqlite3 module
- pandas: comparison with SQL
- DB Browser for SQLite
- SQLite SELECT
- SQLite date and time functions
- SQLite aggregate functions
- RFC 9562: UUIDs (version 5 = name-based)